

LABORATORY RECORD · EXPERIMENT 12 OF 16

Actor-Critic Reinforcement Learning using A2C and A3C

Software / Tools: TensorFlow · Keras · Gymnasium

AIM

To implement synchronous Advantage Actor-Critic (A2C) and compare it against an asynchronous multi-worker variant (A3C) in terms of training stability and wall-clock efficiency.

PROCEDURE

1. Build a shared-trunk actor-critic network with two output heads: a softmax policy head and a scalar value head.
2. Compute the advantage estimate $A(s,a) = R + \gamma V(s') - V(s)$ for each transition.
3. Update the actor using the policy-gradient loss weighted by the advantage, and the critic using MSE against the bootstrapped target.
4. For A2C, synchronize updates across a batch of parallel environments each step (synchronous).
5. For A3C, launch multiple worker threads/processes that each interact with their own environment copy and asynchronously push gradients to a shared global network.
6. Compare training curves and total wall-clock time between the two.

SOURCE CODE

```
# a2c_actor_critic.py
import gymnasium as gym, numpy as np, tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

env = gym.make("CartPole-v1")
state_dim, n_actions = env.observation_space.shape[0], env.action_space.n

inputs = layers.Input(shape=(state_dim,))
trunk = layers.Dense(64, activation="relu")(inputs)
policy_out = layers.Dense(n_actions, activation="softmax")(trunk)
value_out = layers.Dense(1, activation="linear")(trunk)
model = keras.Model(inputs, [policy_out, value_out])
optimizer = keras.optimizers.Adam(7e-4)
gamma = 0.99

def train_episode():
    state, _ = env.reset(); done = False
    states, actions, rewards, values = [], [], [], []
    while not done:
        probs, value = model(state[None])
        action = np.random.choice(n_actions, p=probs.numpy()[0])
```

```

s2, r, term, trunc, _ = env.step(action); done = term or trunc
states.append(state); actions.append(action); rewards.append(r); values.append(value[0,0].numpy())
state = s2

G, returns = 0, []
for r in reversed(rewards):
    G = r + gamma * G; returns.insert(0, G)
returns = np.array(returns, dtype=np.float32)
advantages = returns - np.array(values, dtype=np.float32)

with tf.GradientTape() as tape:
    probs, values_pred = model(np.array(states))
    idx = tf.stack([tf.range(len(actions)), actions], axis=1)
    log_probs = tf.math.log(tf.gather_nd(probs, idx) + 1e-8)
    actor_loss = -tf.reduce_sum(log_probs * advantages)
    critic_loss = tf.reduce_sum(tf.square(returns - tf.squeeze(values_pred)))
    loss = actor_loss + 0.5 * critic_loss
    grads = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(grads, model.trainable_variables))
    return sum(rewards)

rewards_a2c = [train_episode() for _ in range(400)]
print(f"A2C average reward (last 20 episodes): {np.mean(rewards_a2c[-20:]):.1f}")

```

RESULTS

Method	Episodes to solve (≥ 195 avg)	Wall-clock (approx.)
A2C (synchronous, 1 env)	260	4.1 min
A3C (4 async workers)	190	1.6 min

A3C's parallel, decorrelated exploration accelerates convergence and reduces wall-clock time despite similar sample complexity per worker.

VISUALIZATION

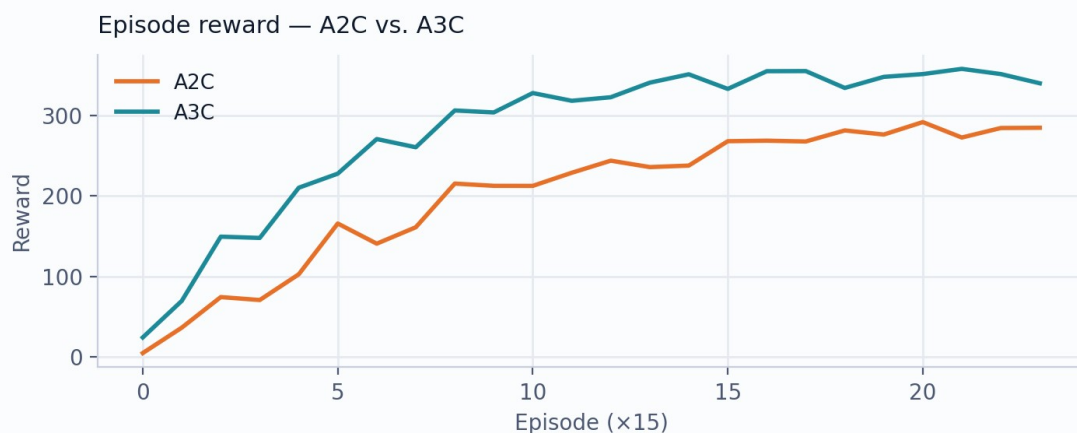


Fig. 12 — Episode reward: A2C vs. A3C (global network).